

開始使用事件導向 Cloud Run functions

來源：<https://codelabs.developers.google.com/codelabs/getting-started-cloud-run-functions-event-driven?hl=zh-tw>

步驟數：8

如何開始使用事件驅動型 Cloud Run 函式

目錄

1. [1. 簡介](#)
2. [2. 設定環境變數並啟用 API](#)
3. [3. 建立儲存空間值區和服務帳戶](#)
4. [4. 建立及部署函式](#)
5. [5. 建立事件](#)
6. [6. 測試函式](#)
7. [7. 恭喜！](#)
8. [8. 清除所用資源](#)

1. 簡介

總覽

Cloud Run functions 是一種部署工作負載的新方式，可使用熟悉的 GCF 事件處理範例和函式簽章。Cloud Run functions 不會使用我們預先設定的建構程序和部署設定，而是讓您直接控管在 Cloud Run 上建立的基礎服務。

透過 Cloud Run functions，我們提供簡單的 Cloud Run 來源部署 UX，讓開發人員使用 Cloud Run 設定，全權控管工作負載。

本節將說明如何在 Node.js 中部署事件驅動函式。您將部署函式，每當物件在 Google Cloud Storage bucket 中完成時，就會觸發該函式。

本程式碼研究室會在下列範例中使用 nodejs 範例。不過，您可以使用所選語言的 Cloud Functions 第 2 代程式碼範例：

- [Python 函式範例](#)
- [Node.js 函式範例](#)
- [Go 函式範例](#)
- [Java 函式範例](#)
- [PHP 函式範例](#)
- [Ruby 函式範例](#)
- [.NET 函式範例](#)

課程內容

- 如何部署事件驅動型 Cloud Run 函式，在物件上傳至 GCS bucket 時觸發
 - 如何建立具備適當角色的服務帳戶，以便接收 Cloud Storage 的事件並叫用 Cloud Run 函式
-

步驟 2 · 約 0 分鐘

2. 設定環境變數並啟用 API

更新 gcloud CLI

您必須安裝最新版本的 gcloud CLI，才能完成本程式碼研究室。如要更新 CLI，請執行

```
gcloud components update
```

啟用 API

開始進行本程式碼研究室之前，請先啟用數個 API。本程式碼研究室需要使用下列 API。執行下列指令即可啟用這些 API：

```
gcloud services enable run.googleapis.com \
```

```
cloudbuild.googleapis.com \
storage.googleapis.com \
artifactregistry.googleapis.com
```

設定環境變數

您可以設定本程式碼實驗室全程都會使用的環境變數。

```
PROJECT_ID=<YOUR_PROJECT_ID>
REGION=<YOUR_REGION, e.g. us-central1>
```

```
gcloud config set project $PROJECT_ID
SERVICE_NAME=crf-event-codelab
BUCKET_NAME=$PROJECT_ID-$SERVICE_NAME
TRIGGER_NAME=$SERVICE_NAME-trigger
```

步驟 3 · 約 0 分鐘

3. 建立儲存空間值區和服務帳戶

建立儲存空間值區

您可以執行下列指令來建立 Cloud Storage bucket：

```
gsutil mb -l us-central1 gs://$BUCKET_NAME
```

建立服務帳戶

在本範例中，您將建立具備必要 EventArc 權限和 Cloud Run 叫用者角色的服務帳戶，以便接收 Cloud Storage 的事件並叫用 Cloud Run 函式。

首先，請建立服務帳戶。

```
PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --  
format='value(projectNumber)')  
  
SERVICE_ACCOUNT="cloud-run-functions"  
SERVICE_ACCOUNT_ADDRESS=$SERVICE_ACCOUNT@$PROJECT_ID.iam.gserviceaccount.com  
  
gcloud iam service-accounts create $SERVICE_ACCOUNT \  
--display-name="Cloud Run functions Eventarc service account"
```

接著，將專案的 Eventarc 事件接收者角色 (roles/eventarc.eventReceiver) 授予與 Eventarc 觸發條件相關聯的服務帳戶，讓觸發條件可以接收事件提供者的事件。

```
gcloud projects add-iam-policy-binding $PROJECT_ID \  
--member serviceAccount:$SERVICE_ACCOUNT_ADDRESS \  
--role=roles/eventarc.eventReceiver
```

接著，將 Cloud Run 叫用者角色授予服務帳戶，讓該帳戶可以叫用函式。

```
gcloud projects add-iam-policy-binding $PROJECT_ID \  
--member serviceAccount:$SERVICE_ACCOUNT_ADDRESS \  
--role=roles/run.invoker
```

步驟 4 · 約 0 分鐘

4. 建立及部署函式

首先，請建立原始碼的目錄，然後 `cd` 到該目錄。

```
mkdir ../$SERVICE_NAME && cd $_
```

接著，請建立 `package.json` 檔案，並加入以下內容：

```
{
  "dependencies": {
    "@google-cloud/functions-framework": "^3.0.0"
  }
}
```

接著，請建立 `index.js` 檔案，並加入以下內容：

```
const functions = require("@google-cloud/functions-framework");

// Register a CloudEvent callback with the Functions Framework that will
// be triggered by Cloud Storage.
functions.cloudEvent("helloGCS", (cloudEvent) => {
  console.log(`Event ID: ${cloudEvent.id}`);
  console.log(`Event Type: ${cloudEvent.type}`);

  const file = cloudEvent.data;
  console.log(`Bucket: ${file.bucket}`);
  console.log(`File: ${file.name}`);
  console.log(`Metageneration: ${file.metageneration}`);
  console.log(`Created: ${file.timeCreated}`);
  console.log(`Updated: ${file.updated}`);
});
```

現在您可以執行下列指令，部署 Cloud Run 函式：

```
gcloud beta run deploy $SERVICE_NAME \
  --source . \
    --function helloGCS \
    --region $REGION \
    --no-allow-unauthenticated
```

請注意下列事項：

- 使用「`--source`」旗標，告知 Cloud Run 將函式建構為可執行的容器型服務
- 使用 `--function` 標記 (新) 將新服務的進入點設為要叫用的函式簽章
- (選用) 使用 `--no-allow-unauthenticated`，禁止公開叫用函式

執行下列指令，即可查看新服務 `crf-event-codelab`：

```
gcloud beta run services describe $SERVICE_NAME
```

5. 建立事件

我們可以建立 Eventarc 觸發條件，在 Google Cloud Storage 中完成物件時，將訊息傳送至函式：

```
BUCKET_REGION=$REGION

gcloud eventarc triggers create $TRIGGER_NAME \
  --location=$REGION \
  --destination-run-service=$SERVICE_NAME \
  --destination-run-region=$BUCKET_REGION \
  --event-filters="type=google.cloud.storage.object.v1.finalized" \
  --event-filters="bucket=$BUCKET_NAME" \
  --service-account=$SERVICE_ACCOUNT_ADDRESS
```

請注意下列事項：

- gcs-function-trigger 是觸發條件的名稱
- crf-event-codelab 是部署函式的 Cloud Run 服務名稱
- 使用 `--event-filters` 標記時，請勿在 bucket 名稱中使用 `gs://` 前置字串。

如需使用 Eventarc 從 Cloud Storage 設定觸發程序服務的詳細教學課程，請參閱 Cloud Run 說明文件：

<https://cloud.google.com/run/docs/tutorials/eventarc>

6. 測試函式

部署完成後，您會看到服務網址。如要叫用函式，您需要傳送經過驗證的要求，並附上您的身分識別權杖，或是具有 Cloud Run 叫用者角色的主體身分識別權杖，如下所示：

```
# get the Service URL
SERVICE_URL="$(gcloud run services describe $SERVICE_NAME --region us-central1 --
format 'value(status.url)')"

# invoke the service
curl -H "Authorization: bearer $(gcloud auth print-identity-token)" -X GET
$SERVICE_URL
```

部署函式並建立觸發條件後，我們現在可以叫用函式。

建立檔案並上傳至 Cloud Storage bucket。您可以使用 Cloud 控制台網頁介面或 gsutil CLI 工具執行這項操作，例如：

```
echo "hello world" > test.txt
gsutil cp test.txt gs://$BUCKET_NAME
```

檔案上傳成功後，系統會產生事件，而函式會列印物件的一些基本資訊，例如檔案名稱。您可以在 Cloud 控制台的函式記錄項目中找到這項輸出內容。或者，您也可以使用 gcloud CLI 查詢這項輸出內容：

```
gcloud logging read "resource.labels.service_name=$SERVICE_NAME AND textPayload:
File" --format=json
```

您應該會看到下列輸出內容

```
"textPayload": "File: test.txt"
```

步驟 7 · 約 0 分鐘

7. 恭喜！

恭喜您完成本程式碼研究室！

建議您參閱 [Cloud Run functions](#) 說明文件

涵蓋內容

- 如何部署事件驅動型 Cloud Run 函式，在物件上傳至 GCS bucket 時觸發
 - 如何建立具備適當角色的服務帳戶，以便接收 Cloud Storage 的事件並叫用 Cloud Run 函式
-

8. 清除所用資源

為避免產生意外費用 (例如，如果這個 Cloud Run 服務的叫用次數不慎超過[免費層級的每月 Cloud Run 叫用次數配額](#))，您可以刪除 Cloud Run 服務，或刪除在步驟 2 中建立的專案。

如要刪除 Cloud Run 服務，請前往 Cloud Run Cloud Console (<https://console.cloud.google.com/run/>)，然後刪除您在本程式碼研究室中建立的 `crf-event-codelab` 服務。

如要刪除整個專案，請前往 <https://console.cloud.google.com/cloud-resource-manager>，選取您在步驟 2 中建立的專案，然後選擇「刪除」。刪除專案後，您必須在 Cloud SDK 中變更專案。如要查看所有可用專案的清單，請執行 `gcloud projects list`。
